CropShield 6 — Manus Task Brief

Copy-paste this when starting a new Manus task

> Continue work on **CropShield 6**, a farmer-focused crop-health workspace.
>
> **Repository:** `Zinbas/cropshield-6`
>
> **Project type:** Managed Manus WebDev full-stack project with React 19, Vite, Tailwind 4, Express, tRPC, Drizzle ORM, MySQL/TiDB, Manus authentication, managed storage, AI scan analysis, weather integration, and Google Maps.
>
> **Primary objective:** Implement and polish the requested feature end-to-end in the existing project. Work directly in the imported repository; do not rebuild the application from scratch.
>
> **Required workflow:**
>
> 1. Inspect the current project, existing routes, related components, database schema, and tests before editing.
> 2. Read the relevant Manus skills and the WebDev project instructions before planning implementation.
> 3. Reuse existing components, especially `DashboardLayout`, `MapView`, tRPC procedures, database helpers, and existing UI tokens.
> 4. Implement frontend, backend, database, and tests together where the feature requires them.
> 5. Keep the UI mobile-first, accessible, and consistent with the existing CropShield design.
> 6. Handle loading, empty, error, and unavailable-integration states honestly; never fabricate data, coordinates, diagnoses, reviews, or testimonials.
> 7. Run `pnpm check`, `pnpm test`, `pnpm build`, and relevant E2E tests.
> 8. Inspect the live preview, fix any runtime or visual regressions, and restart the preview if another session has changed the project.
> 9. Push all completed changes to `Zinbas/cropshield-6` on `main`.
> 10. Save a managed WebDev checkpoint after verification and report the checkpoint link, GitHub commit, tests, and any known limitations.
>
> **Do not stop at analysis or a partial implementation.** Make reasonable low-risk assumptions and record them in the final report. Ask only when a missing decision would materially change behavior, permissions, data, billing, or an irreversible external action.

Current CropShield product context

CropShield supports farmer profiles and locations, crop records, image-based AI scan analysis, scan history, follow-up cases, regional disease and pest risk alerts, local weather guidance, verified experts, approved agricultural stores, and administrator workflows.

The current showcase dataset contains **10 showcase farmers, 5 approved drug stores, 7 experts, and 10 approved AI scans**. The administrator regional map uses aggregated approved scan data. The 10 farmers are represented in the Nashik regional aggregate. Experts and stores are directory records and should only be plotted when verified latitude and longitude values exist; do not invent coordinates.

Risk alerts are predictive early warnings with a 7–15 day outlook. They must be labelled as **Predicted Risk** or **Potential Threat**, not confirmed disease cases. Farmer-level locations must not be exposed in aggregate administrator views.

Important files to inspect first

Area	File or directory
Main application pages	`frontend/src/pages/Home.tsx`
Shared map integration	`frontend/src/components/Map.tsx`
Regional risk engine	`frontend/src/lib/regionalRisk.ts`
Heatmap transformer	`frontend/src/lib/regionalHeatmap.ts`
API procedures	`backend/routers.ts`
Database helpers	`backend/db.ts`
Database schema	`database/schema.ts`
Existing unit tests	`frontend/src/**/*.test.ts`, `backend/*.test.ts`
Browser tests	`e2e/cropshield.spec.ts`
Playwright configuration	`e2e/playwright.config.ts`
Guided setup and operations	`docs/PREVIEW_SETUP_GUIDE.md`, `docs/CROPSHIELD_HANDOFF.md`

Fast implementation plan

Phase 1 — Establish the baseline

Run:

```
git status --short
git log -3 --oneline
pnpm check
pnpm test
```

Confirm the managed preview URL and current checkpoint. If the task says the preview is stale or another session has pushed changes, call the managed `webdev_restart_server` tool before making assumptions about the running application.

Phase 2 — Trace the feature vertically

Before coding, identify:

- The current route and navigation entry.
- The page/component that owns the UI.
- The tRPC query or mutation that supplies data.
- The database table and helper behind that procedure.
- Existing tests covering neighboring behavior.
- Any required integration skill, such as Maps, storage, AI, or scheduling.

Prefer the smallest end-to-end change that fits the current architecture. Do not introduce a second API client, a parallel state-management system, or duplicate map loader.

Phase 3 — Implement with production-safe behavior

- Use existing tRPC hooks for data access.
- Add database schema and migrations only when genuinely required.
- Keep UTC timestamps at the API/database boundary and localize them only for display.
- Store media in managed storage and retain only references in the database.
- Add explicit error and empty states.

- Add stable accessible labels and selectors for important interactive elements.
- Ensure every new route has a clear back/navigation path on mobile and desktop.
- Keep pure helpers in utility modules so Fast Refresh warnings are avoided.

Phase 4 — Validate in layers

Run the focused tests first, then the complete suite:

```
pnpm check
pnpm test
pnpm build
pnpm test:e2e
```

For a non-default preview URL:

```
E2E_BASE_URL="https://your-preview-url" pnpm test:e2e
```

Use environment variables for E2E credentials; never commit passwords or API keys:

```
E2E_ADMIN_EMAIL="..." E2E_ADMIN_PASSWORD="..." pnpm test:e2e
```

Check the actual browser experience at both desktop and mobile widths. For maps, verify the map container, fallback overlay, legend, controls, click/tap behavior, and fullscreen behavior. For scan history, verify loading, empty, error, search/filter, expansion, recommendation progress, and image/reference rendering as applicable.

Phase 5 — Synchronize and hand over

Before finishing:

```
git status --short
git add <intended-files>
git commit -m "<clear change description>"
git push github main
git ls-remote github refs/heads/main
```

Then save a managed WebDev checkpoint. The final report should state:

- What changed.
- The managed preview/checkpoint link.
- The GitHub commit and branch.
- Exact validation results.
- Any known limitations or follow-up work.

Definition of done

The task is complete only when all applicable items are true:

- [ ] The requested user-visible behavior works in the live preview.
- [ ] Mobile and desktop layouts remain usable.
- [ ] Existing authentication and role boundaries are preserved.
- [ ] Backend and database behavior is implemented where needed.
- [ ] Loading, empty, error, and unavailable-integration states are handled.
- [ ] Relevant unit and browser tests are added or updated.
- [ ] `pnpm check` passes.
- [ ] `pnpm test` passes.
- [ ] `pnpm build` passes.
- [ ] Relevant E2E tests pass.
- [ ] No secrets, generated artifacts, or local logs are committed.
- [ ] Changes are pushed to GitHub `main`.
- [ ] A managed checkpoint is saved.
- [ ] The final response includes the preview/checkpoint link, commit, test results, and limitations.

Useful follow-up request format

When adding another feature, provide the request in this format for fastest execution:

```
Feature:
[One sentence describing the user-visible outcome]

Users and route:
[Farmer/admin/expert; route or navigation area]

Data:
[Existing data to reuse, new fields, or whether mock/showcase data is acc

Interactions:
[Tap/click, filters, forms, fullscreen, upload, navigation, mobile behavi

Acceptance criteria:
- [Concrete visible result]
- [Error/empty/loading behavior]
- [Accessibility or responsive requirement]
- [Test expectation]

Constraints:
[Do not change; integrations; privacy; performance; deadline]

Finish by implementing, testing, pushing to GitHub main, and saving a Webl
```

Security and data rules

Never paste secrets into the task brief, source code, GitHub, screenshots, or chat. Use the configured Manus secret manager or environment variables. Do not expose farmer-level location data in aggregate maps. Do not create fake medical/agricultural certainty: AI scan findings and regional risk signals must be clearly distinguished from confirmed diagnoses.